■ **Theory Content to be taught :**

1. Unit 6 : Graph.
2. Basic Terminologies and Representation
3.
4.
5.

■ **Specific Objectives :**

1.
2.
3.
4.
5.

■ **Theory Content :**

❶ Introduction to Graph :-

A Graph is a non-linear data structure consisting of a set of Vertices (nodes) and edges (links) that connect pairs of vertices.

Unlike arrays, linked list, stacks, and queues (which are linear data structures), graph represent relationship between objects.

Graphs are widely used in:

✓ social networks ✓ Google Maps

✓ Computer Networks.
✓ Web pages (Hyperlinks)
✓ operating systems etc.

• Defination :-

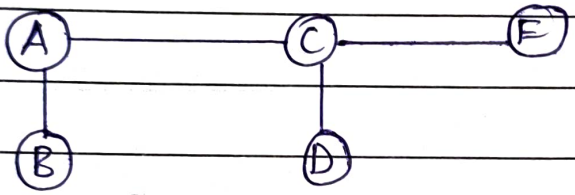
A Grap 'G' is mathematically represented as -

$$G = (V, E)$$

Where -

- V = set of vertices (Nodes)
- E = set of Edges (connections)

Ex :-



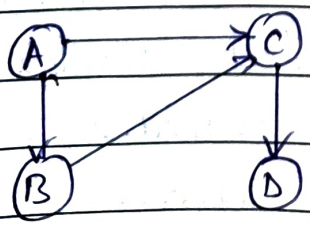
$V(G) = \{ A, B, C, D, E \}$.

$E(G) = \{ (A,B), (A,C), (C,D), (C,E) \}$.

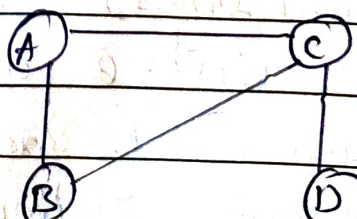
• A graph can be directed or undirected.

✓ in an undirected graph, the edges do not have any direction associated with them. i.e. if an edge is drawn between nodes A and B, then the nodes can be traversed from A to B as well as B to A.

Ex :-



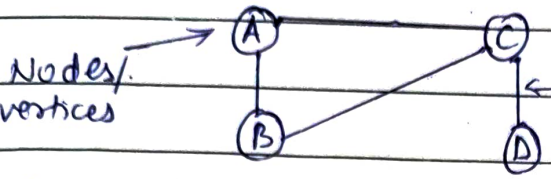
Directed Graph



Undirected Graph.

Graph Terminologies :-

① **Graph :-** Graph is a collection of nodes and edges which connects nodes in the graph.



Generally, the graph G is represented as, G=(V,E) where V is the set of vertices and E is the set of edges.

② **Vertex / Node :-**

A vertex / node is a point / data element in the graph.

In above graph, A, B, C, D are 4 vertices / nodes.

③ **Edge :-**

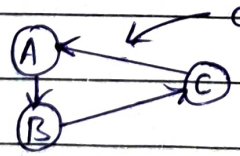
An edge is the connection between two vertices.

Ex :- E(G) = {(A,B), (A,C), (B,C), (C,D)}.

④ **Directed Graph :-**

Edges with direction are called as directed graph.

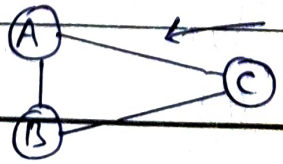
Ex :-



⑤ **Undirected Graph :-**

Edges with no direction are called as undirected graph.

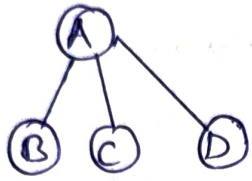
Ex :-



⑥ Degree of a vertex / node :-

The no. of edges connected to a vertex is called it's degree.

Ex :-



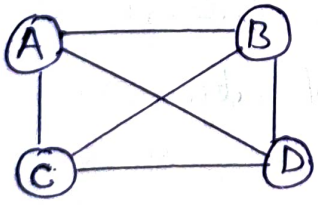
NODE	DEGREE
A	3
B	1
C	1
D	1

⑦ Complete Graph :-

An undirected graph, in which every vertex has an edge to all other vertices is called a complete Graph.

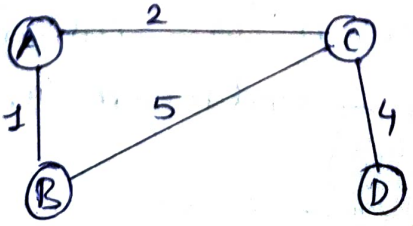
A complete graph with 'N' edges has $\frac{N*(N-1)}{2}$ edges.

Ex :-



⑧ weighted Graph :- A weighted graph is a graph in which edges are assigned some value.

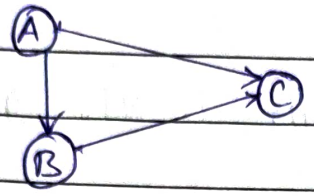
Ex :-



(9) In-degree :-

The In-degree of a vertex/ node is the number of edges entering (pointing toward) that vertex/node.

Ex :-



Indegree(A) = 0.
Indegree (B) = 1.
Indegree (C) = 2.

(10) Out-degree :-

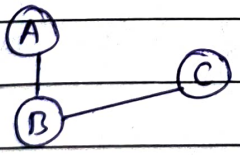
The out degree of a vertex/node is the number of edges leaving (originating from) that vertex/ node.

Ex :- For above graph - Outdegree (A) = 2.
Outdegree (B) = 1.
Outdegree (C) = 0.

(11) Adjacent Node :-

Two vertices/nodes connected by an edge are called adjacent nodes.

Ex :-



Adjacent Nodes are - (A, B) and (B, C).

(12) Successor :-

If V_1 is adjacent to V_2, V_1 is called successor of V_2.

(13) Predecessor :- If V_1 is adjacent to V_2, V_2 is called predecessor of V_1.

(14) path :-

A sequence of vertices / nodes connected through edges.

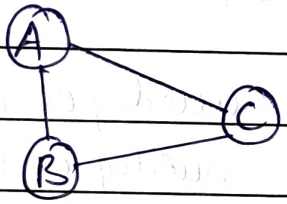
(15) Length of a path :-

The length of a path is the number of edges on it.

(16) cycle :-

A path that starts and ends at the same vertex / node.

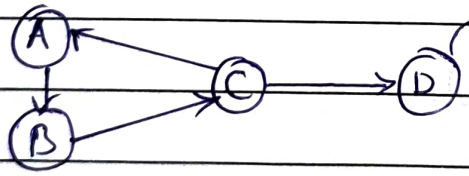
Ex :-



cycle ⇒ A - B - C - A.

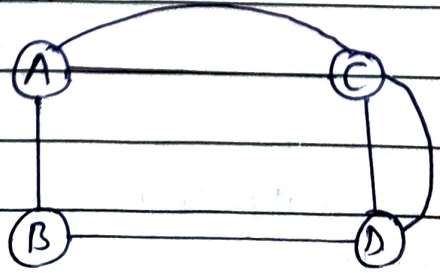
(7) self Edge or self loop :-

An edge of the from (V,V) is known as self edge or self loop.



← self edge.

(18) Multigraph :- A graph with multiple occurrences of the same edge is known as multigraph.



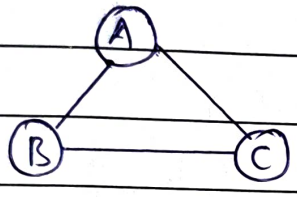
(19) sink node :-

A node having only incoming edges & no outgoing edges.

(20) Isolated Node :-

A node which is not adjacent to any other node.

Ex :-



→ 'D' is called Isolated node.



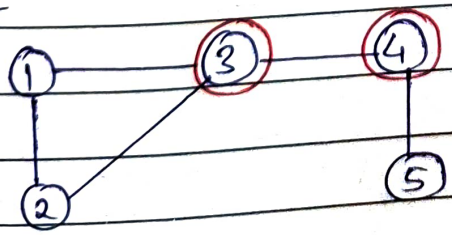
(21) pendant Node :-

A node having in-degree 1 and out degree is zero.

(22) Articulation point :-

A node in an undirected connected graph is an articulation point if removing it disconnects the graph.

Ex :-



→ '3' and '4' are articulation point



→ (1) and (2) are articulation point

5.

■ **Theory Content :**

① Representation of Graphs

There are two common ways of storing graph in Computer's Memory.

1) sequential representation by using an Adjacency Matrix.

2) Linked representation by using an Adjacency List.

1) Adjacency Matrix Representation :-

An adjacency matrix is used to represent which nodes are adjacent to one another.

- By defination, two nodes are said to be adjacent if there is an edge connecting them.
- In a directed graph 'G', if node 'v' is adjacent to 'u', then there is definitely an edge from u to v.

- For any graph 'G' having 'n' nodes, the adjacency matrix will have the dimension of n×n.

- The adjacency matrix 'A' of graph G is a two-dimensional array of n×n elements where 'n' is the no. of nodes / vertices.

- The entries of 'A' (matrix) are defined as -
  - A[i][j] = 1, if there exist an edge between vertices i and j in graph G.
  - A[i][j] = 0, if no edge exist between i and j.

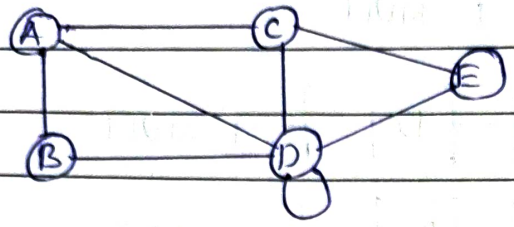
★ Adjacency matrix of Undirected Graph :-

From the adjacency matrix, it is very easy to calculate the degree of vertices of an undirected graph.

- The degree of vertex 'i' is the no. of 1's

in it's row. (or the sum of row 'i').

Ex:-

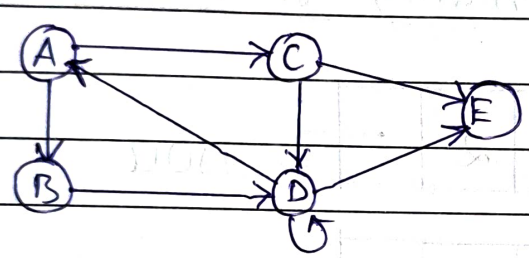


A =

	A	B	C	D	E
A	0	1	1	1	0
B	1	0	0	1	0
C	1	0	0	1	1
D	1	1	1	1	1
E	0	0	1	1	0

* Adjacency matrix of Directed Graph:-

Ex:-



A =

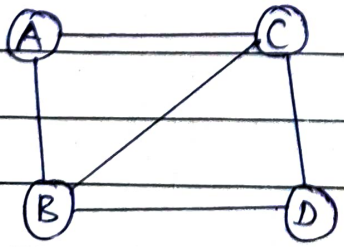
	A	B	C	D	E
A	0	1	1	0	0
B	0	0	0	1	0
C	0	0	0	1	1
D	1	0	0	1	1
E	0	0	0	0	0

2) Adjacency list Representation :-

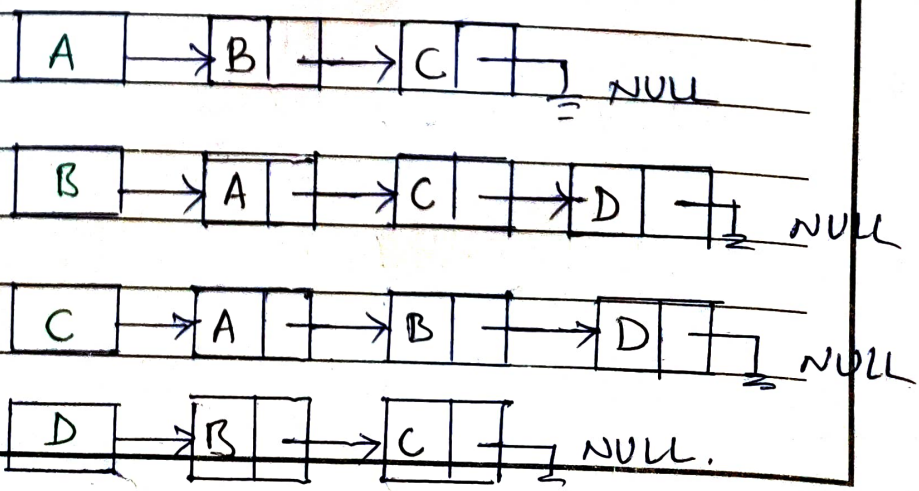
An adjacency list is used in the linked representation to store the graph in the computers memory.

* Adjacency list representation of Undirected Graph.

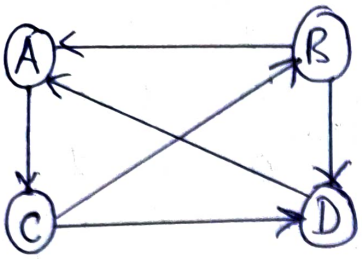
Ex:-



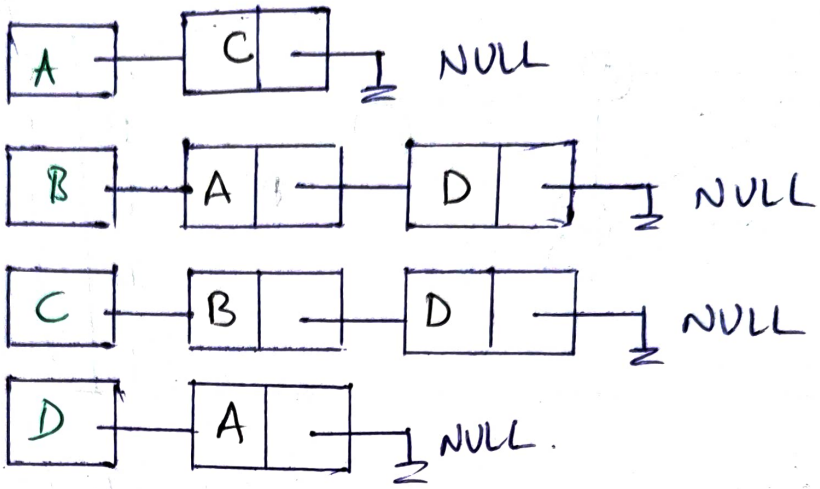
Graph.



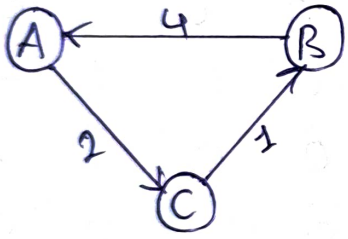
* Adjacency list representation of Directed Graph :-

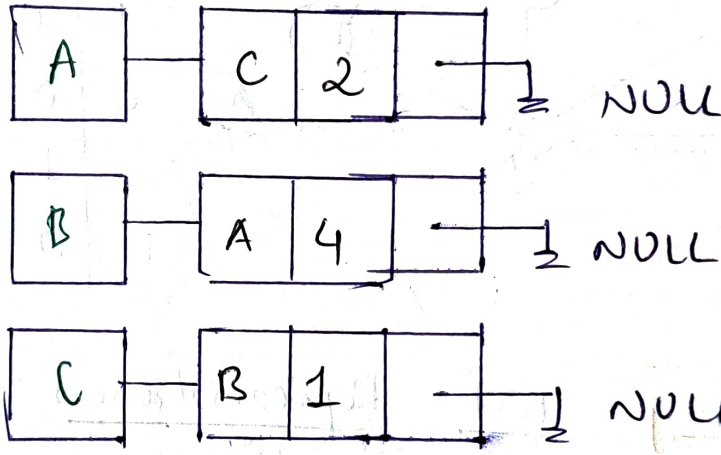


Graph.



* Adjacency list representation of weighted Graph :-





■ **Theory Content to be taught :**

1. Graph search and traversal algorithms and
2. complexity analysis.
3.
4.
5.

■ **Specific Objectives :**

1.
2.
3.
4.
5.

■ **Theory Content :**

Graph traversal :-

Graph traversal means visiting every vertex (node) of a graph exactly once in a systematic order.

There are two major graph traversal algorithms.

1. Breadth first search (BFS)
2. Depth first search (DFS).

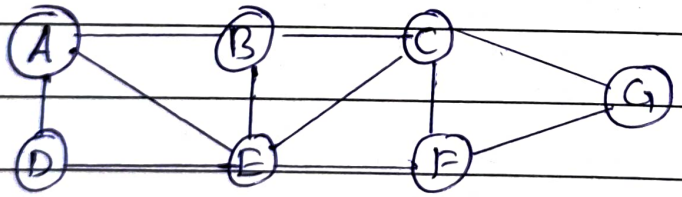
1. Breadth First Search (BFS) :-

Breadth First search (BFS) is a graph traversal algorithm that visits nodes level by level. We use Queue data structure with max size of total number of nodes in the graph to implement BFS traversal.

- **Working Principle :-**

1. start from the source node.
2. Mark it as visited.
3. Insert into Queue.
4. Remove front node.
5. Visit all unvisited neighbors.
6. Insert neighbors into Queue.
7. Repeat until Queue becomes empty.

Example :- Consider the following graph.



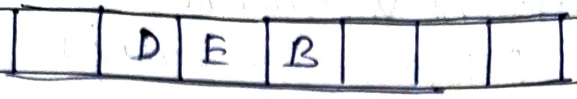
Step 1 : ✓ Select the vertex A as a starting point (visit A). - Insert A into Queue.



Step 2 : ✓ visit all adjacent vertices of A which are not visited (D, E, B)

✓ Insert newly visited vertices into the

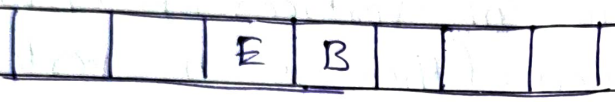
queue and delete A from the queue.



⇒ Queue.

step 3:

- ✓ visit all adjacent vertices of D which are not visited (there is no vertex).
- ✓ Delete D from the queue.



⇒ Queue.

step 4:

- ✓ visit all adjacent vertices of E which are not visited (C, F).
- ✓ Insert newly visited nodes into the Queue and delete E from the queue.



⇒ Queue.

step 5:

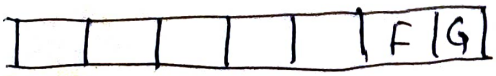
- ✓ visit all adjacent vertices of B which are not visited (there is no vertex).
- ✓ Delete B from the queue.



⇒ Queue.

step 6: ✓ visit all adjacent vertices of C which are not visited. (G).

- ✓ insert newly visited vertex into queue and delete C from the queue.



⇒ Queue.

step 7 :-

✓ visit all adjacent vertices of F which are not visited (there is no vertex).

✓ Delete F from queue.



⇒ Queue.

step 8 :-

✓ Visit all adjacent vertices of G which are not visited (there is no vertex).

✓ Delete G from Queue.



⇒ Queue.

- Queue becomes empty. So stop BFS process.

o BFS Complexity analysis :-

- suppose, V = no. of nodes, E = no. of edges.

Each vertex/node visited once. O(V).

Each edge checked once O(E).

Total time = O(V+E).

space complexity =

Queue stores at most V vertices/nodes.

= O(V).

■ **Theory Content to be taught :**

1. Graph search and traversal algorithms and
2. complexity analysis.
3.
4.
5.

■ **Specific Objectives :**

1.
2.
3.
4.
5.

■ **Theory Content :**

2) Depth first search (DFS) :-

Depth first search (DFS) explores as deep as possible before backtracking. We use stack data structure with max size of total no. of vertices in the graph to implement DFS traversal.

Use following steps :-

Step 1: Define a stack of size total no. of vertices in the graph.

step 2: select any vertex as starting point for traversal. Visit that vertex and push it on to the stack.

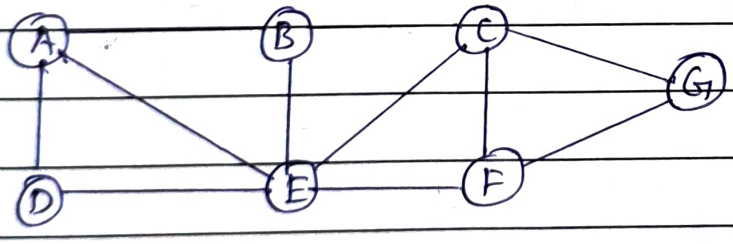
step 3: Visit any one of the non-visited adjacent vertices of a vertex which is at the top of stack and push it on to the stack.

step 4: Repeat step 3 until there is no new vertex to be visited from the vertex which is at the top of the stack.

step 5: When there is no new vertex to visit then use backtracking and pop one vertex from the stack.

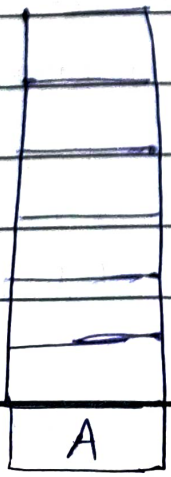
step 6: Repeat steps 3, 4 and 5 until stack becomes empty.

Ex:- Consider following graph.



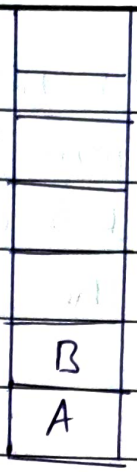
step 1: ✓ select the vertex A as a starting point (visit A).

✓ push 'A' on to the stack.



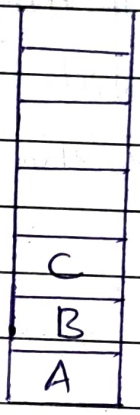
stack.

step 2: ✓ visit any adjacent vertex of A which is not visited. (B).
✓ push newly visited vertex B on to the stack.



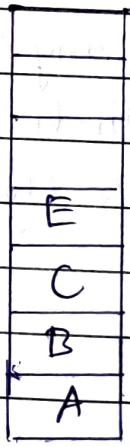
stack.

step 3: ✓ visit any adjacent vertex of B which is not visited (C).
✓ push C on to the stack.



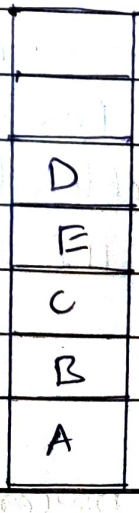
stack.

step 4: ✓ visit any adjacent vertex of C which is not visited. (E).
✓ push E on to the stack.



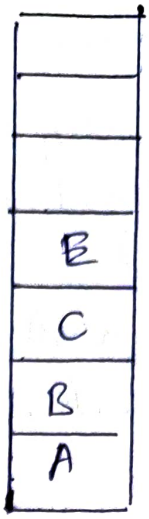
stack.

step 5: ✓ visit any adjacent vertex of E which is not visited. (D).
✓ push D on to the stack.



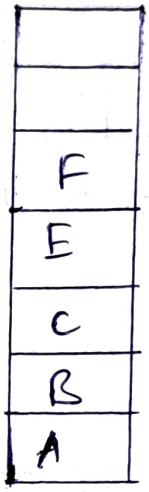
stack.

step 6: ✓ There is no new vertex to be visited from D.
✓ So use back track.
✓ pop D from the stack.



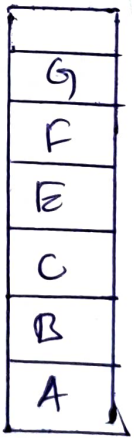
stack

step 7: ✓ visit any adjacent vertex of E which is not visited (F).
✓ push F on to the stack.



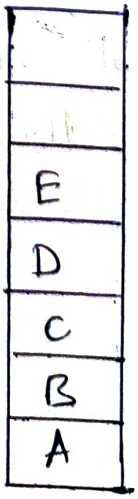
stack.

step 8: ✓ visit any adjacent vertex of F which is not visited (G).
✓ push G on to the stack.



stack

step 9: ✓ There is no new vertex to be visited from G.
✓ So use backtrack.
✓ pop G from the stack.



stack.

Repeat above procedure until stack is empty.

• DFS Complexity analysis :-

✓ Each vertex visited once - O(V).

✓ Each edge visited once - O(E).

Total Time = O(V+E).

space ⇒ no. of vertices / nodes = O(V).

■ **Theory Content to be taught :**

1. Warshall's Algorithm.
2.
3.
4.
5.

■ **Specific Objectives :**

1.
2.
3.
4.
5.

■ **Theory Content :**

① Transitive Closure of a Directed Graph :-

In graph theory, a directed graph contains vertices (nodes) and directed edges (arrows).

Sometimes, we need to find whether a path exists between two vertices, even if there is no direct edge between them.

To find all possible paths between vertices, we use "Transitive closure.".

° Defination :-

The transitive closure of a directed graph is a graph that shows whether a path exists between every pair of vertices.

In simple words, Transitive closure tells us whether we can reach one vertex from another vertex by following the directed edges.

It includes,

- ✓ All the orignal edges of the graph.
- ✓ All additional connections possible through intermediate vertices.

° Example of Transitive Closure :-

Consider the following directed graph :-



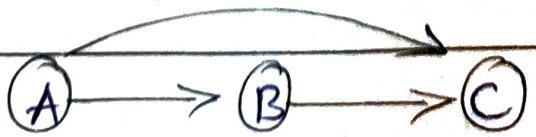
[orignal graph]

The graph contains three vertices : A, B & C.

The edges are :-

- ✓ A → B
- ✓ B → C.

Here, there is no direct edge from A to C. But we can reach C from A through B. Therefore, a new connection A → C is added in the transitive closure.



[transitive closure graph].

final edges :-

✓ A → B

✓ A → C

✓ A → C (New connection).

Thus, the transitive closure shows all possible reachable connections.

- Adjacency Matrix of Transitive closure :-

An adjacency matrix is a table used to represent the connections between vertices.

✓ Write 1 if a path exists between two nod

✓ Write 0 if no path exists between two nodes.

Consider following graph :-



- Initial Adjacency Matrix :-

a[] [] =

	A	B	C
A	0	1	0
B	0	0	1
C	0	0	0

- final Transitive Closure Matrix :-

a[] [] =

	A	B	C
A	0	1	1
B	0	0	1
C	0	0	0

Here, A → C is added because there is a path A → B → C.

⑥ Warshall's Algorithm :-

Warshall's Algorithm is used to find the transitive closure of a directed graph.

It checks all vertices one by one as intermediate vertices.

Steps of Warshall's Algorithm :-

1. Create an adjacency matrix of the given directed graph.
2. Select the first vertex as an intermediate vertex.
3. Check whether a new path is possible through that vertex.
4. If a new path found, change 0 to 1.
5. Repeat the same process for all vertices.
6. Display the final transitive closure matrix.

- **formula** :- $A[i][j] = A[j][j]$ OR $(A[i][k]$ & $A[k][j])$

Where :-
- ✓ i = Starting vertex.
- ✓ j = Destination vertex.
- ✓ K = Intermediate vertex.

Example :- Consider following graph.



Edges :- A → B and B → C.

Initial Adjacency Matrix :-

A[ ][ ] =

	A	B	C
A	0	1	0
B	0	0	1
C	0	0	0

Warshall's Algorithm :-

1. Start.
2. Read the adjacency matrix.
3. Repeat for k = 1 to n.
4. Repeat for i = 1 to n.
5. Repeat for j = 1 to n.
6. Apply the formula :
   A[i][j] = A[i][j] OR (A[i][k] AND A[k][j]).
7. Display the final matrix.
8. Stop.

Step-by-step Solution :-

Step 1 :- Take A as an intermediate vertex. (K = 1).

Check whether any new path is possible through vertex A.

✓ There is no incoming edge to A.
✓ Therefore, no new path is found.
✓ The matrix remains unchanged.

Matrix after step 1:

A[][] =

	A	B	C
A	0	1	0
B	0	0	1
C	0	0	0

Step 2: Take B as an intermediate vertex (K = 2).

Check whether any new path is possible through vertex B.

Look at the graph:

✓ A → B ✓ B → C.

Therefore, we can travel from A to C through B.

New path ⇒ A → B → C.

Change the matrix value from 0 to 1 at position (A, C).

Matrix after step 2:

A[][] =

	A	B	C
A	0	1	1
B	0	0	1
C	0	0	0

Step 3:- Take C as an intermediate vertex. (K = 3).

check whether any new path is possible through vertex C.

- ✓ C has no outgoing edges.
- ✓ Therefore, no new path is found.
- ✓ The matrix remains unchanged.

Matrix after step 3:-

A[][] =	A	B	C
A	0	1	1
B	0	0	1
C	0	0	0

Final Transitive Closure Matrix :-

T =			
	0	1	1
	0	0	1
	0	0	0

Thus, the transitive closure of the given graph is obtained using Warshall's algorithm.

- Time Complexity :- $O(n^3)$, where 'n' is the number of vertices.